

HW6

MSML606

Author: Chenhongshu Yu

UID: 116610971

Scope

This notebook documents all required answers for:

- Part I: Problems 1 (edge cases testing, limitations of recursive DFS)
- Part II: Problems 1-3 (all three questions)

Statement: The use of external resources is clearly cited. Please refer to the bold words in my submission. If no citation is found for any specific part of my submission, it means "No external sources were used; all ideas are my own or from course lecture slides".

Part I: Number of Islands

Check the Detailed Implementations (BFS, DFS, DFS_Stack) in Homework6.py file.

Problem 1. Implement the following methods to solve the problem:

- Breadth-First Search (BFS) using a queue
- Depth-First Search (DFS) using recursion
- Depth-First Search (DFS) using a stack (iterative)
- Test with edge cases (e.g., empty grids, single-element grids, all-land or all-water)
- Mention any limitations of recursive DFS in your analysis

Generate

+ Code

+ Markdown

Edge Case Testings:

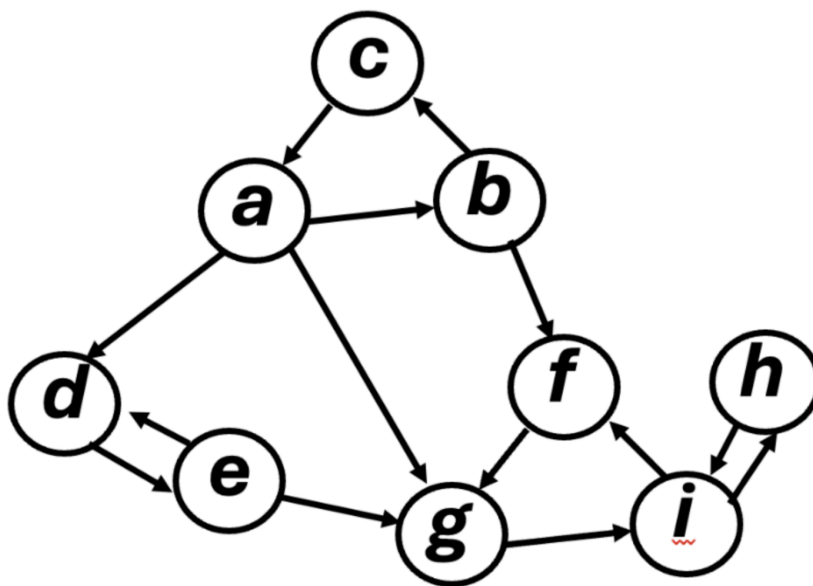
For edge case testing, I tested empty grids, single-element grids, all-land grids, and all-water grids.

- An empty grid such as `[]` or `[[]]` should return 0 because there are no land cells.
- A single-cell grid like `[['1']]` should return 1, while `[['0']]` should return 0.
- An all-land grid should return 1 because all land cells are connected, and an all-water grid should return 0 because no islands exist.

Limitation of recursive DFS

Recursive DFS can fail on very large grids because Python has a recursion depth limit. If an island is too large, the recursive calls may cause a `RecursionError`.

Part II: Graph Traversal (DFS)



Problem 1

Perform a DFS on a graph above, starting at a vertex not used in the lecture example. Choose a different edge exploration order so that the traversal order is different from the one in the lecture notes.

In the lecture example, the starting vertex is chosen at **a**. For my DFS traversal of the graph above, I choose to start with vertex **h**.

Starting DFS at **h**, we can get **h** $\rightarrow$ **i** $\rightarrow$ **f** $\rightarrow$ **g**. Because at **g**, the only outgoing edge is **g** $\rightarrow$ **i**, but **i** has already been discovered, so DFS backtracks.

Since the graph is directed and not every vertex is reachable from **h**, we continue the DFS with the remaining unvisited vertices, using alphabetical order for the next unvisited vertex, which is vertex **a**. For vertex **a**, it has **a** $\rightarrow$ **d** and **a** $\rightarrow$ **b**.

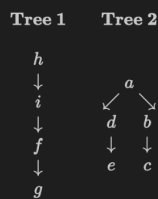
We start with **a** $\rightarrow$ **d**. We can get **a** $\rightarrow$ **d** $\rightarrow$ **e**. Similarly, at **e**, both outgoing edges go to already discovered vertices, so DFS backtracks to **a**. Next, we continue from **a** $\rightarrow$ **b**. We can get **a** $\rightarrow$ **b** $\rightarrow$ **c**. Similarly, at **c**, the edge **c** $\rightarrow$ **a** goes to an already discovered vertex, so DFS finishes.

Therefore, since the DFS finished, we can get the resulting DFS discovery order was **h, i, f, g, a, d, e, b, c**.

Problem 2

- Draw the DFS forest or trees resulting from your traversal
- Label the edges of the original graph with their respective classes (tree, back, forward, cross)

Using the DFS order we chose in Problem 1, the DFS forest has two trees, because not every vertex is reachable from **h**. We have **h** $\rightarrow$ **i** $\rightarrow$ **f** $\rightarrow$ **g** and (**a** $\rightarrow$ **d** $\rightarrow$ **e**, **a** $\rightarrow$ **b** $\rightarrow$ **c**). The DFS forest can be drawn as:



Citation: Used AI tool for getting the drawing trees in latex format in jupyter notebook file.

Labels of the edges:

- Tree edges: **h** $\rightarrow$ **i**, **i** $\rightarrow$ **f**, **f** $\rightarrow$ **g**, **a** $\rightarrow$ **d**, **d** $\rightarrow$ **e**, **a** $\rightarrow$ **b**, **b** $\rightarrow$ **c**
- Back edges: **i** $\rightarrow$ **h**, **g** $\rightarrow$ **i**, **e** $\rightarrow$ **d**, **c** $\rightarrow$ **a**
- Cross edges: **a** $\rightarrow$ **g**, **e** $\rightarrow$ **g**, **b** $\rightarrow$ **f**
- Forward edges: None

Problem 3

Create a table for each vertex **v** with:

- $\pi(v)$: predecessor
- $d(v)$: discovery time
- $f(v)$: finish time

As mentioned in problem 1, the DFS starts at **h**, we get the DFS order in **h, i, f, g, a, d, e, b, c**.

Table:

Vertex <i>v</i>	$\pi(v)$	$d(v)$	$f(v)$
<i>h</i>	NIL	1	8
<i>i</i>	<i>h</i>	2	7
<i>f</i>	<i>i</i>	3	6
<i>g</i>	<i>f</i>	4	5
<i>a</i>	NIL	9	18
<i>d</i>	<i>a</i>	10	13
<i>e</i>	<i>d</i>	11	12
<i>b</i>	<i>a</i>	14	17
<i>c</i>	<i>b</i>	15	16